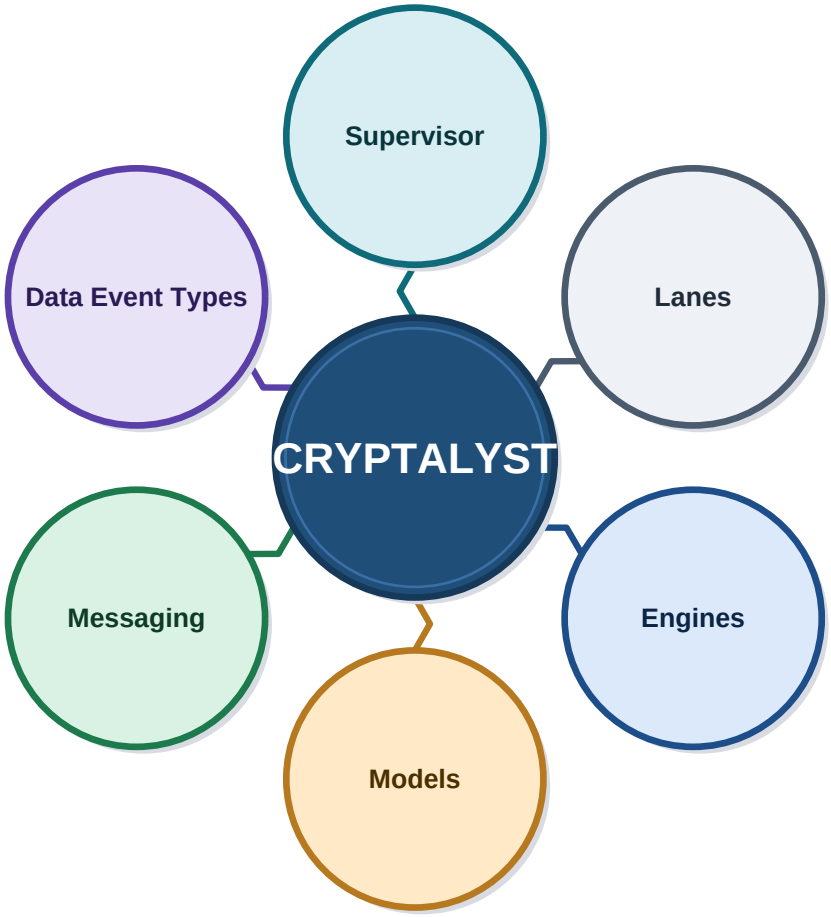
CRYPTALYST



Supervisor · Lanes · Engines · Models · Messaging · Data Event Types

Platform Version 2.0 · Document Revision 2

Designed by Andrew Preston · **© TekVolt Systems 2026**

Field	Value
Document	CryptAlyst — Supervisor, Lanes, Engines, Models, Messaging, Data Event Types
Platform Version	2.0
Document Revision	2
Status	living reference; the 22-topic registry is normative
Audience	engineers, quants, risk, and anyone asked to change the platform
Scope	the running paper-trading platform: bus, engines, models, books
Designed by	Andrew Preston
Copyright	© TekVolt Systems 2026

Reading order. Chapter 1 defines the engine/model/book layers and the decision weighting; Chapter 2 the supervisors and lanes; Chapter 3 the engines; Chapter 4 the gate chain and how a BUY or SELL is decided; Chapter 5 the models; Chapter 6 the books; Chapter 7 a worked example of a signal becoming a trade — each with Mermaid flow diagrams. Chapter 8 is the bus reference (architecture, registry, producers, subscribers); Chapter 9 lists the known gaps.

Contents

Chapter 1 · Overview — engines, models and books 3
Chapter 2 · Supervisors & lanes 4
Chapter 3 · Engines — the running processes 6
Chapter 4 · Gates — from rule to allowed trade 11
Chapter 5 · Models — the rules that score 13
Chapter 6 · Books — the paper accounts 17
Chapter 7 · A signal becomes a trade, end to end 20
Chapter 8 · The message bus 21
Chapter 9 · Known gaps — models not listening 27

Chapter 1 · Overview — engines, models and books

Engine = a running process (the supervisor launches one `python3 -u <file>.py`). **Model** = a strategy/rule inside an engine — one engine can run many models, and `--only` narrows it. **Book** = the paper-trading account for one model: its equity, open positions and closed trades. One book per model. A model's numbers land on its book.

When a model has finished computing, its numbers are not a trade — they are a **signal** that then passes a chain of gates and weights before the book opens a position.

1 · the model scores. The model's rule reads its inputs and returns a **side** (buy/sell) and a **score** (its conviction). **2 · the forecast.** The 7-agent forecaster blends momentum/volume/volatility/reversion/ma/breakout/pressure into one **probability (0–1)** plus an **expected move**. **3 · the TA.** tad supplies signal votes (EMA cross, RSI, Donchian). **4 · the OI gate.** the price×OI quadrant multiplies the score, strategy-aware. **5 · the gates.** forecast_min (0.45), min_expected_move, allow_counter_trend, and the higher-timeframe trend gate (ext_gate) each veto or admit. **6 · the risk gate.** risk.allow checks max_positions, confidence and signal age. **7 · the entry.** if every gate passes, the book opens a position, sized by the weighting below.

How the weighting works. The forecast is the primary weight: its blended probability must clear forecast_min to enter at all. Conviction sizing scales capital with the number of confirmations — `risk_for(base, conf) = base × (0.6 + 0.2 × min(conf, 4))`. The OI quadrant is a **strategy-aware multiplier** on the score (momentum prefers confirmation, reversion prefers divergence), and `size_mult` (now 3.0) scales the final notional. No single number is decisive — the decision is the product of the model score, the forecast probability, and these multipliers.

Chapter 2 · Supervisors & lanes

The **supervisor** (`supervisor.py`) is the platform's shepherd, not its trader. It never decides *what* to trade — it decides *which engines are allowed to run* and keeps them alive. A **lane** is one engine's slot in the fleet: the process it runs in, the code it owns, and the inputs it is allowed to read. The relationships are fixed and simple: **an engine is 1-to-1 with a lane**, and **an engine is 1-to-many with models** (each model is one book).

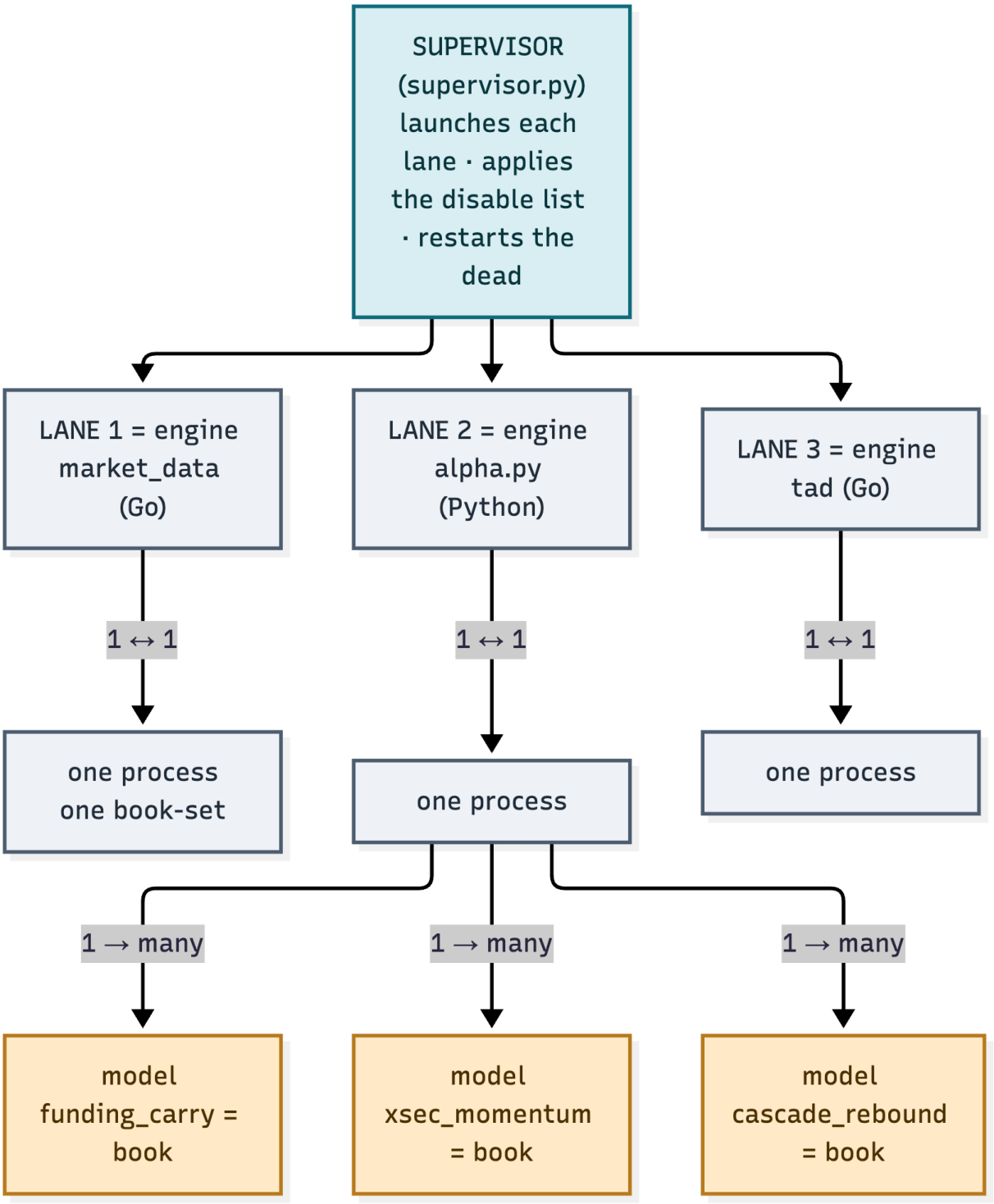


Figure 2.1 — the supervisor allocates lanes; each lane is one engine (1:1); each engine hosts many models (1:many), one book per model.

What the supervisor actually does, in order: it reads the **disable list** (`state/disabled.json`) and refuses to launch any disabled engine (A6 — a re-enable is a human act); it launches each enabled

engine as its own process; it **watches** them (a crash or a hang is restarted by the watchdog); and it grades each book (trades, return, streak) for the leaderboard. Because the disable check happens again inside `start()`, a relaunch cannot silently revive a disabled book — that hole was closed after the best-5 enforcement.

The 1:1 / 1:many split is the whole point of the layering. A lane is the unit of **lifecycle** (launch, disable, restart); a model is the unit of **logic** (one rule); a book is the unit of **state** (equity, positions, trades). Changing a model's rule never touches the lane; killing a lane never touches another engine's models.

Chapter 3 · Engines — the running processes

An **engine** is one running process — a Go binary or a `python3 -u file.py`. It owns a lane: the supervisor launches it, it subscribes to the topics it needs, and it loops. One engine can host several models; `--only` narrows it to one. `alpha.py` runs three models (funding_carry, xsec_momentum, cascade_rebound), `models10.py` runs funding_fade, and `crypto_strategies.py` runs model_stack. Some engines are Go (market_data, tad, forecast, backend, recorder), some are Python (the books, signals, feeds) — they interoperate only through the bus and the envelope.

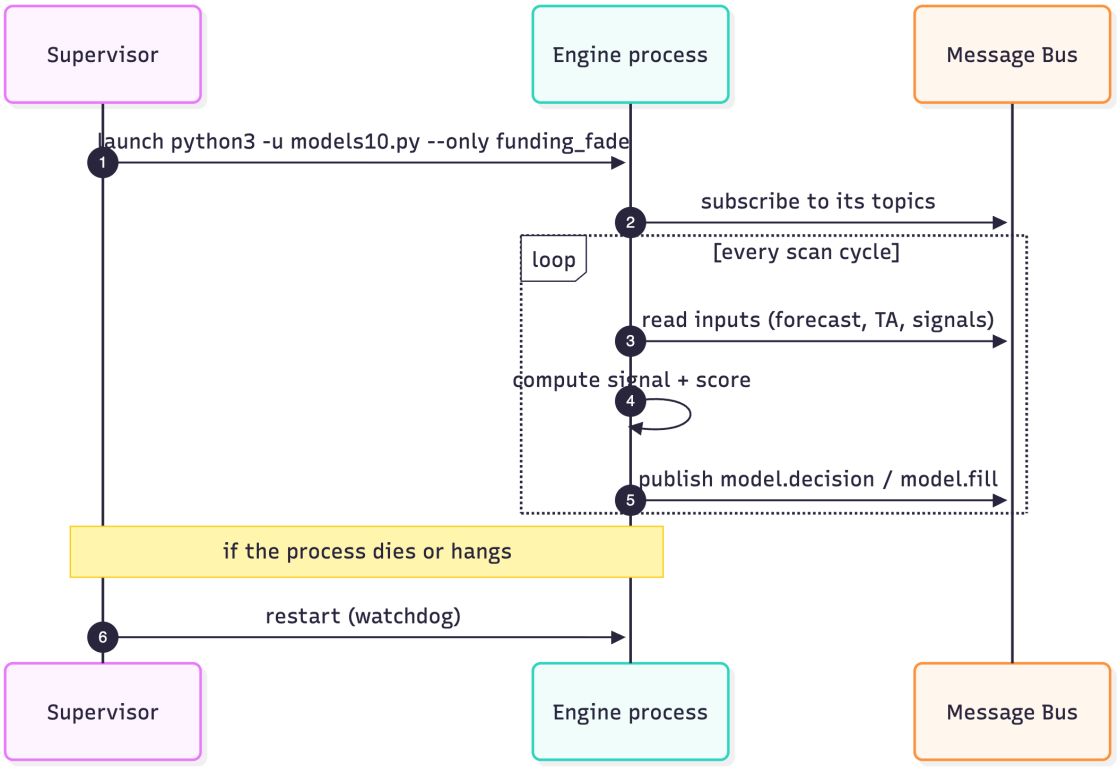


Figure 3.1 — an engine's life: launch, subscribe, loop, and the supervisor's watchdog restart when it dies.

The lifecycle matters because it is **stateless across restarts**: an engine's book state is rebuilt from disk, not carried in memory. The supervisor keeps the fleet alive, but it never decides *what* trades — it only decides *which engines are allowed to run* (the disable list) and restarts the ones that crash.

3.1 · How an engine discovers its topics

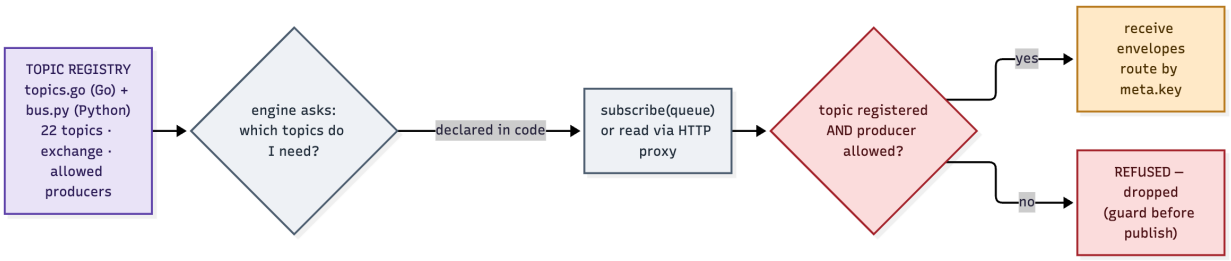


Figure 3.2 — topic discovery: the registry is the single source of truth; an engine declares its needs in code, and the bus refuses anything unregistered or unauthorised.

There is no magic. The **topic registry** (`core/internal/topics/topics.go` in Go, `lib/bus.py` in Python) is the one list of every topic, its exchange, and its allowed producers. An engine 'knows' its topics because they are **declared in its code**: it either subscribes to a queue for the topic, or reads the latest via an HTTP proxy (forecast :8790, tad :8792, signal_bus :8793, master :8799). On publish, the bus re-checks the registry — an unregistered topic or an unauthorised producer is refused before it reaches

the wire. That is the discovery process: *declare, then validate against the registry.*

3.2 · Python and Go in one fleet

A book is Python; the market-data feed and the forecasters are Go. They are not 'all in one engine' — they are separate processes that meet at the **envelope**, a language-agnostic JSON shape `{topic, key, producer, data, version, seq, ts}`. A Go producer (market_data, tad, forecast) marshals its struct to that JSON and publishes it; a Python consumer receives the body and, if it has no `topic` field, wraps it into the same envelope shape (`bus.py:_decoding`). The language is invisible at the boundary: `models10.py` calls the Go forecast over HTTP and reads a Go-written envelope without ever importing Go. The only contract is the JSON shape and the topic name — which is why the two registries must stay in sync.

3.3 · What an engine reads — inputs and their shapes

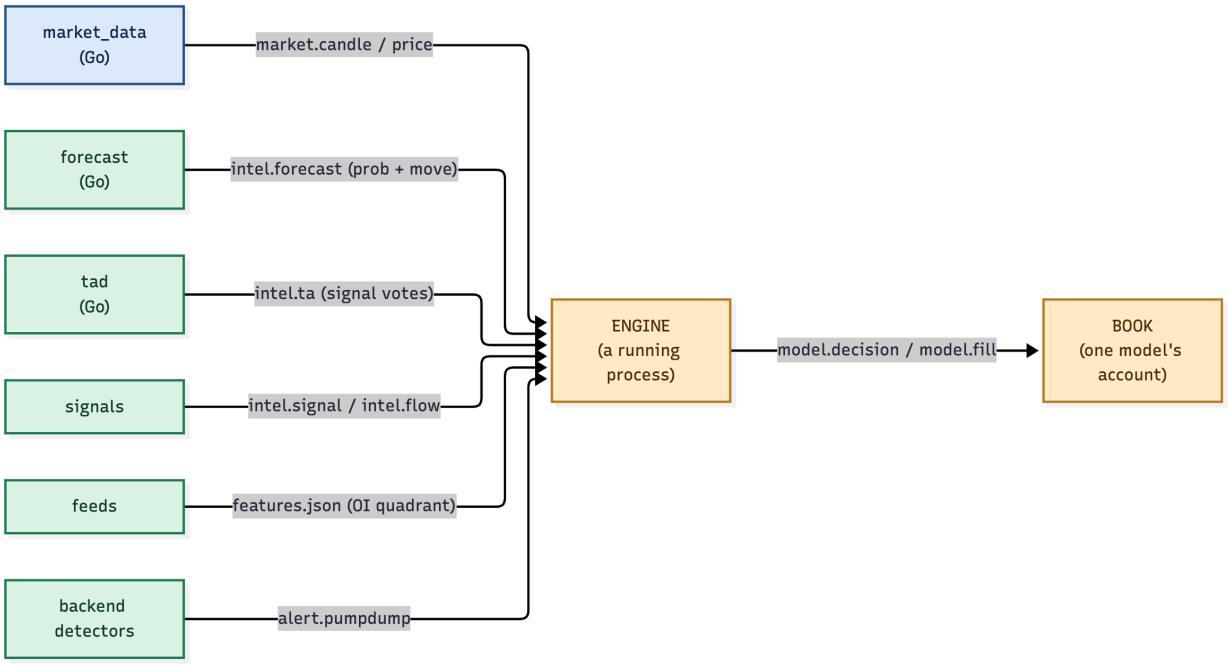


Figure 3.3 — every input an engine can read, and the one thing it writes: a decision on the book.

No engine talks to Binance directly except **market_data**. Everything else reads the bus topics or the REST/file proxies. The inputs, with their exact shapes, are:

Input	Via	Shape
`market.candle`	klines	`array of {t, o, h, l, c, v} — one bar per row`
`intel.forecast`	REST :8790	`{views:[{agent, side, prob, move_pct}], prob, move_pct, horizon, bar_age_s}`
`intel.ta`	REST :8792 / ta_live.json	`{symbol, signals:[{kind, side}], count}`
`intel.signal`	REST :8793	`{symbol, score, side, flow_conf}`
`intel.flow`	REST :8793	`{symbol, flow_conf} — conviction for sizing`
`features.json (OI)`	file	`{symbol, oi_chg_5m, oi_chg_15m, oi_chg_1h, px_chg_15m, oi_quadrant}`
`alert.pumpdump`	REST :8787	`{symbol, kind(PUMP/DUMP), severity, score, metrics:{burst_z,...}}`

An engine reads only the inputs its models need — a funding book ignores the OI quadrant, a momentum book ignores funding. Each input is fetched fresh every cycle, and its age is carried through the gates so a stale value becomes a refusal, never a silent trade.

3.4 · K-lines

A **K-line** (candlestick) is one bar of OHLCV: **o**pen, **h**igh, **l**ow, **c**lose, **v**olume, stamped with a timestamp. market_data serves them per symbol and interval — `/klines?symbol=BTCUSDT&interval=1m&limit=120` returns 120 one-minute candles. Everything derived — EMA, RSI, ATR, Donchian, the forecast, the momentum rank — is computed from these bars. The intervals matter: 1m for fast TA and liquidation proxies, 15m for the funding_fade rule and the forecast blend, 4h for the BTC regime drift every alt inherits. On a fresh boot market_data backfills 200 × 1m and 200 × 15m bars per symbol, so the platform is never history-starved.

3.5 · The Engine Scan Loop

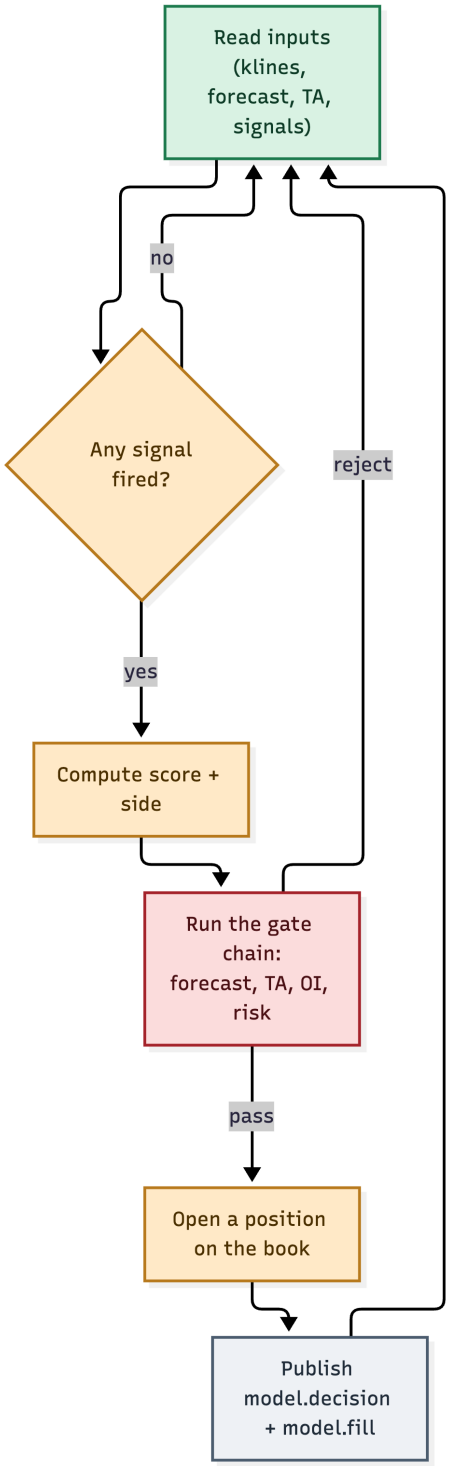


Figure 3.4 — the engine's loop: read, score, gate, trade, publish, repeat.

The loop is the same shape in every engine: pull fresh inputs, ask each model's rule for a signal, run the gate chain, and — only if every gate passes — open a position and publish the decision. The gate chain is shared (risk.py, tactics.py), so no engine can invent its own risk policy.

3.6 · The OI quadrant

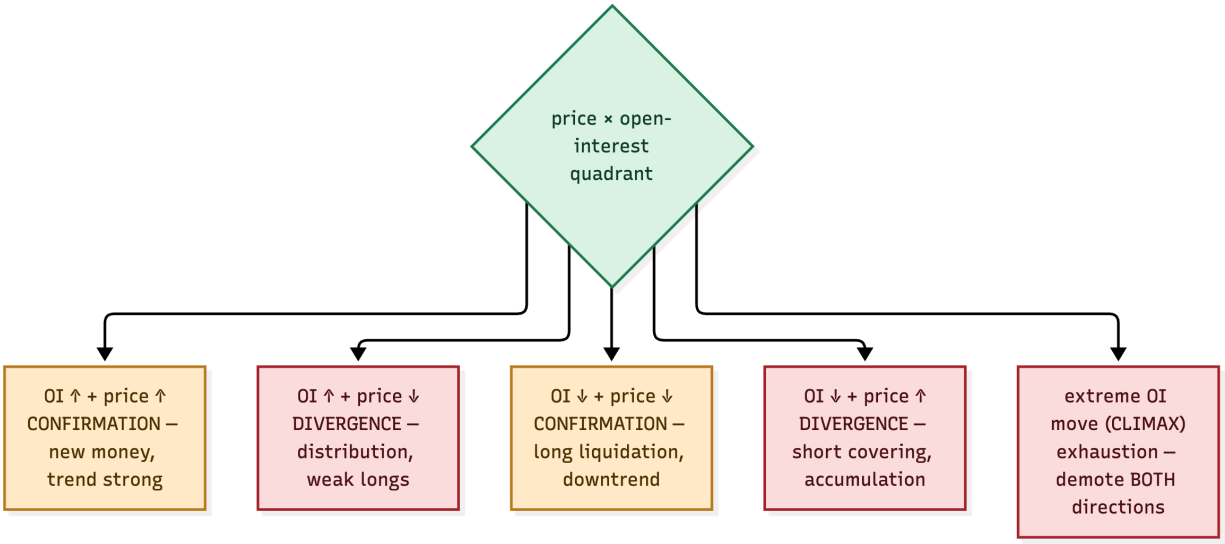


Figure 3.5 — the price × open-interest quadrant: where price and OI move together, or diverge, and the climax that demotes both directions.

Open interest is the number of open contracts; it tracks whether money is entering or leaving. Crossing it with price gives four regimes. **Confirmation** (OI and price move together) is a genuine trend — new money pushing. **Divergence** (they move apart) is a weak move — distribution or short covering. The **climax** case (an extreme OI move) is exhaustion and demotes **both** directions. The engine carries this quadrant on each decision (`oi_quadrant`), and the OI gate turns it into a strategy-aware score multiplier — momentum prefers confirmation, reversion prefers divergence.

3.7 · The engine roster

Engine	Role	Publishes
`market_data (Go)`	the only Binance feed	market.price / candle / depth / mini24h / ticks
`tad (Go)`	technical-analysis daemon	intel.ta, model.evaluate
`forecast (Go)`	7-agent forecaster	intel.forecast
`signals (Python)`	taker ratio + flow confidence	intel.signal / intel.flow / intel.movers
`feeds (Python)`	open-interest + book imbalance	features.json (a file, not a topic)
`market_agent (Python)`	the opportunity board	intel.board + opportunities.json
`alpha (Python)`	funding_carry, xsec_momentum, cascade_rebound	model.decision / fill / pnl
`models10 (Python)`	funding_fade (via --only)	model.decision / fill
`crypto_strategies (Python)`	model_stack (via --only)	model.decision / fill
`churn (Python)`	throughput demo	model.decision / fill
`backend (Go)`	pump/dump + breakout detectors	alert.pumpdump + screen/survey
`recorder (Go)`	the single lake writer	state/lake3 (not a topic)
`portfolio_review (Python)`	grades books + stops losers	review.verdict + disabled.json

Chapter 4 · Gates — from rule to allowed trade

A signal is not a trade. Between the rule's output and an open position sits the **gate chain** — a fixed sequence of checks that every signal must survive, in order. A **no** at any gate drops the signal for that cycle; nothing is retried in a loop. This is the one place the platform refuses risk, and it is shared by every book (risk.py, tactics.py), so no model can invent its own risk policy.

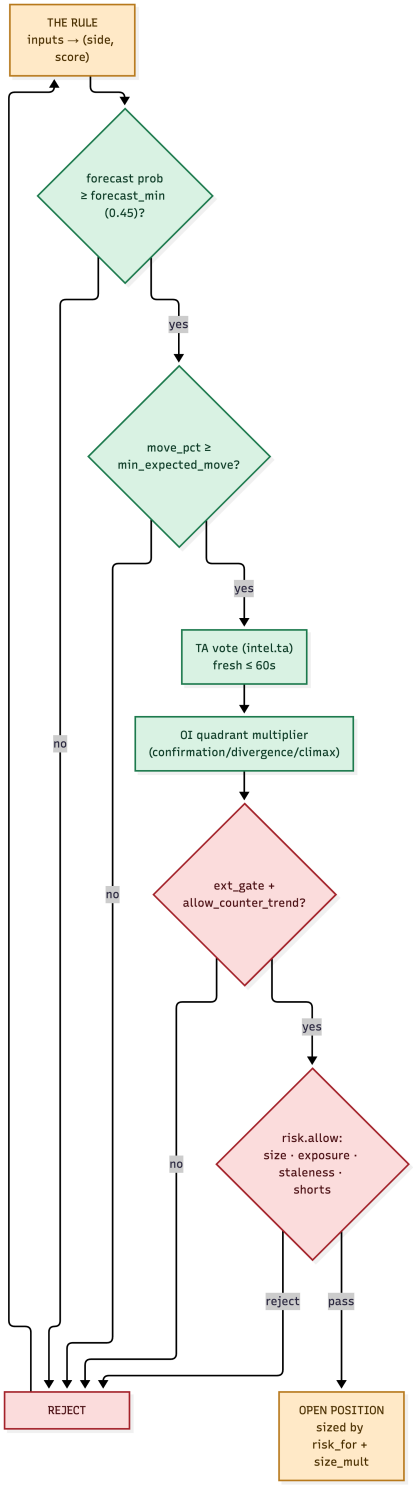


Figure 4.1 — the gate chain. Each gate is a veto; a no sends the model back to the next scan cycle.

4.1 · The rule

The **rule** is the pure function at the head of the chain: it reads the inputs and returns a **side** (`buy` / `sell` / `none`) plus a **score** (its conviction). It has no memory and no opinion about risk — risk is the gates' job. The rule only answers one question: *given these inputs, which way, and how strongly?*

4.2 · How a BUY or SELL is decided, with worked examples

The decision is the rule's answer surviving the gates. Three worked examples:

Example 1 — funding_fade decides SELL. The rule reads `lastFundingRate`. The crowd is paying heavily to stay long, so the reversion is short. fr = +0.10% → above +0.08% → rule returns `("sell", 0.8)`. Then the gates: the forecast must not say strongly buy (prob ≥ 0.45 in the sell direction), the move must clear the fee, the OI quadrant must not be a fresh long-accumulation, and risk.allow must admit the size. If all pass, the book opens a short.

Example 2 — xsec_momentum decides BUY. The rule ranks symbols by 24h return and longs the strongest, shorts the weakest (market-neutral). For a name in the strongest bucket it returns `("buy", rank_score)`. The forecast must agree (prob ≥ 0.45 for buy), and because it is a momentum long, the OI gate *boosts* a confirmation quadrant (OI up + price up) and *demotes* a divergence. If the gates pass, the book buys.

Example 3 — a signal the gates REJECT. funding_fade returns `("sell", 0.9)`, but the forecast returns prob 0.55 *for buy* — the forecast gate vetoes the short (the blend disagrees). No trade. The model re-scores next cycle. This is the whole point: a strong rule answer is overruled by a disagreeing forecast, because the forecast is the primary weight.

4.3 · The forecast, in detail

The forecast is the primary gate. Seven agents each look at the same 1m/15m candle series from a different angle — **momentum** (EMA slope + recent return), **volume**, **volatility**, **reversion**, **moving-average**, **breakout**, and **pressure** — and each returns a probability (a sigmoid of its raw score) plus an expected move. A supervisor blends them, weighting each agent by its historical hit rate (`w = 0.5 + accuracy`, so 0.5–1.5), into one `prob` and one `move_pct`.

Worked blend. Seven agents return probabilities 0.62, 0.58, 0.50, 0.66, 0.55, 0.60, 0.49. With equal weights the blend is ≈ 0.57 — above 0.45, so the forecast admits. But if momentum and breakout (the two with the best hit rate) are the 0.62 and 0.66, their higher weights pull the blend toward 0.60, strengthening the admission. The action is labelled from the blend: `enter` when prob ≥ 0.58 and move ≥ 0.25%, `liquidate` when prob ≤ 0.42, otherwise `hold`. Below MIN_BARS=50 bars the forecaster refuses to answer at all.

Chapter 5 · Models — the rules that score

A **model** is a pure rule: it takes inputs and returns a **side** (buy / sell / none) plus a **score** (its conviction). It has no memory of its own — its memory is the book it writes to. A model never trades directly; it only produces a signal that the gate chain (Chapter 4) then accepts or rejects.

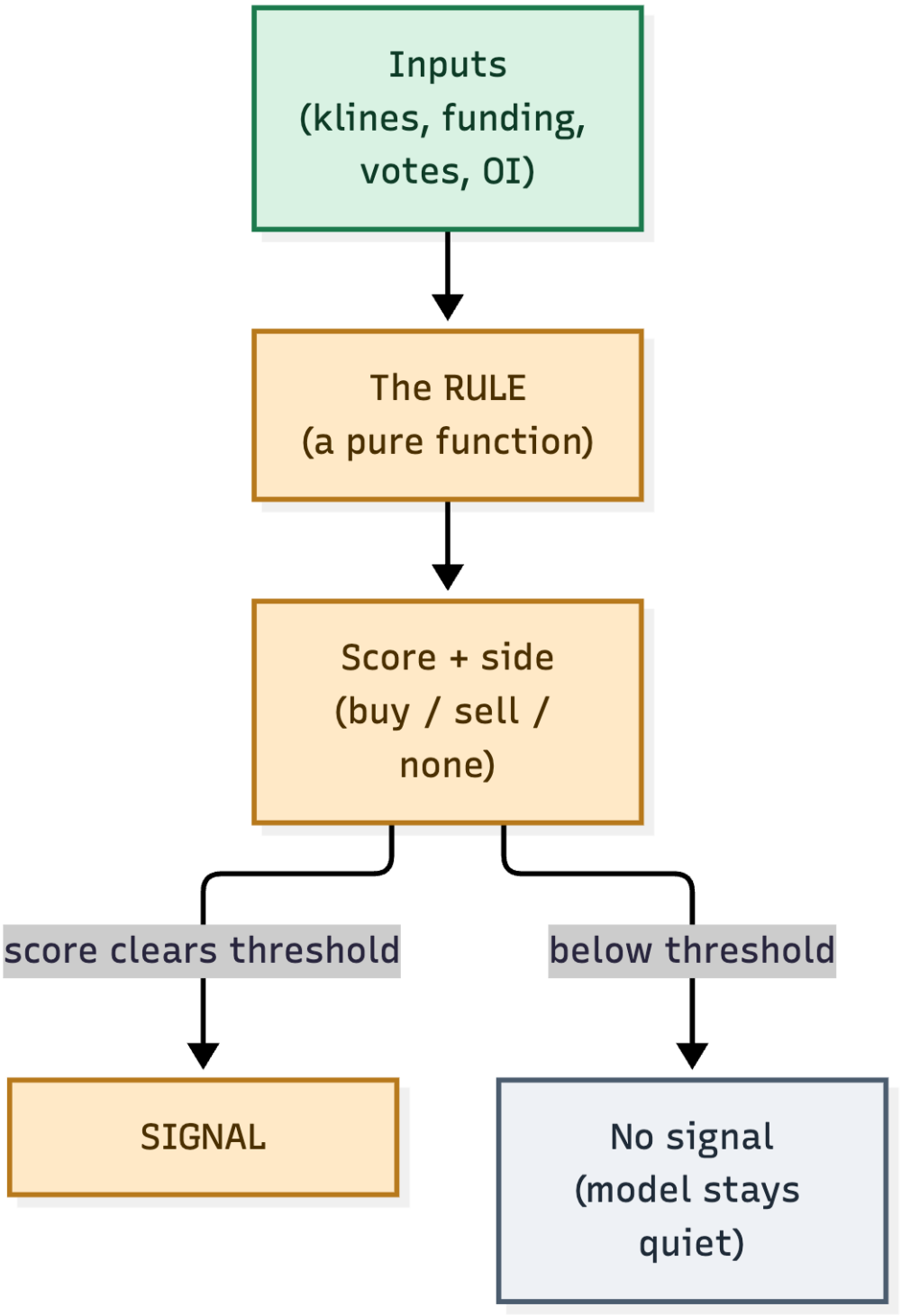


Figure 5.1 — a model is a function: inputs in, a scored signal out. Below the threshold it stays silent.

Every model follows this shape, but the *rule inside the box* is different for each one. funding_fade looks at the funding rate, xsec_momentum looks at 24h returns, model_stack looks at the model votes. What is identical is the output: a scored signal that enters the same gate chain.

5.1 · The models, side by side

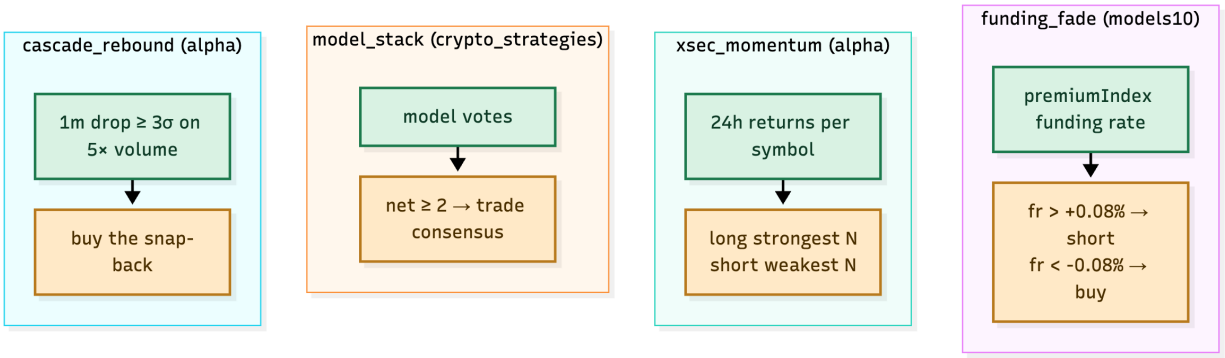


Figure 5.2 — four models, four different inputs, the same output shape: a scored signal.

This is the key to understanding the platform: **the model is the smallest replaceable part**. Swapping a rule changes only what happens inside one box — the inputs, the gates, the sizing and the book are all shared. The owner's best-5 selection chose specific models to keep; the rest are disabled (A6) and influence nothing. Each description below ends with a **Used by** box naming its consumer.

5.2 · Each model in detail

market_data (Go :8796)

How it works — The single process allowed to talk to Binance. It subscribes to the exchange websocket (bookTicker, depth5, kline_1m/15m, miniTicker) and shards the streams — at most 900 per connection — so ~400 symbols are carried on several independent sockets. On startup it BACKFILLS 200 × 1m and 200 × 15m bars per symbol from REST, so the platform is never history-starved after a restart. It serves REST endpoints (/klines, /movers, /streamprices) and publishes the normalised topics.

Best used for — The raw truth everything else derives from. Books must never hit Binance directly — they read these topics or the REST endpoints. The backfill is what made the platform start trading again after a fresh boot.

Used by — every engine — the single raw feed no one else is allowed to touch

tad (Go :8792)

How it works — The technical-analysis daemon. It pulls 1m klines per symbol and computes EMA (9/21/50), RSI14, ATR14, VWAP, a 20-bar Donchian channel and a volume-spike ratio, then emits the signals that actually FIRE — EMA crosses, overbought/oversold, breakouts, VWAP stretch, trend vs EMA50. Since the per-symbol rebuild it recomputes only when a symbol's own bar closes (no batch sweep), and it paces the model-evaluate pulse to the books.

Best used for — Its intel.ta signals are the TA vote in the decision chain — a book reads them as confirmation, and the 60s staleness turns an absent vote into a no-vote. Its model.evaluate pulse is the pacer that should wake each book when a symbol's signal set completes.

Used by — the books (TA confirmation) — and the model.evaluate pacer pulse

forecast (Go :8790)

How it works — Seven agents look at the same 1m series from a different angle — momentum (EMA slope + recent return), volume, volatility, reversion, MA, breakout, pressure — and each returns a probability (a sigmoid of its raw score) plus an expected move. A supervisor BLENDS them into one probability and move estimate, and a grade() feedback loop reweights each agent by whether its past calls made money. It refuses to answer below MIN_BARS=50 (warm-up).

Best used for — The primary weight in a decision: its blended probability must clear forecast_min (0.45) to enter, its move must clear min_expected_move, and consensus_dead closes a losing position only when the blend AND a majority of agents turn against the thesis.

Used by — the gate chain — the primary weight (forecast_min) for every entry

signals (Python)

How it works — Pulls the most-traded names from marketdata, then computes each symbol's taker buy/sell ratio (long/short pressure) and a flow confidence from its recent flow series. It also republishes the movers list so that surface reaches the bus.

Best used for — Its intel.flow feed is the conviction input to sizing (risk_for scales capital with flow confidence); intel.signal is the entry signal for the signal-consuming books.

Used by — the books — flow confidence feeds the conviction sizing

feeds (Python)

How it works — The data the models could not see before. It polls open interest, the order book and the taker ratio, keeps a 1-hour ring buffer of OI and mid-price per symbol, and computes the OI change over 5m/15m/1h plus the price×OI quadrant (confirmation/divergence/accumulation/distribution/climax).

Best used for — Its features.json feeds the OI gate — the quadrant is a strategy-aware score multiplier (momentum prefers confirmation, reversion prefers divergence), and it is the substrate for the F4 with/without-gate replay.

Used by — the OI gate — the quadrant multiplier on each decision

market_agent (Python)

How it works — Watches the liquid universe and scores every name, then ranks the top opportunities and flags the ones above a score threshold. It writes the board to disk AND publishes it to intel.board.

Best used for — The owner-critical opportunity board the owner reviews every cycle — and alpha reads its flagged names to know which symbols to trade.

Used by — the owner's board review — alpha reads its flagged names

alpha · funding_carry, xsec_momentum, cascade_rebound (Python)

How it works — Three fee-viable books, each its own strategy: xsec_momentum is cross-sectional — long the strongest N and short the weakest N by 24h return, rebalanced every 15m, market-neutral so BTC beta cancels and only relative alpha remains. cascade_rebound buys the snap-back after a 1m drop of 3 sigma on 5× median volume (a liquidation proxy without the feed). funding_carry fades a perpetual-funding extreme while spot is flat, collecting the reversion over ~8h. Each reads the board, klines, funding, forecast and TA, then trades through the shared gates.

Best used for — The owner-selected winners. Their model.decision/fill/pnl are the realised PnL the leaderboard and the solver measure. Each book's tunables live in its CFG (tp/sl/hold/risk).

Used by — the owner-selected winners — the leaderboard and the solver measure them

models10 · funding_fade (Python)

How it works — Reads the funding premium (premiumIndex). If the funding rate is above +0.08% it shorts — the crowd is paying heavily to stay long, so the reversion is fadable; below -0.08% it buys. It is gated by the forecast and the higher-timeframe trend, and sized by RISK with the 8%-of-book cap scaled by size_mult.

Best used for — The kept funding book. Its signal is a mean-reversion on the funding premium; the forecast and trend gates stop it fading into a regime that is still trending.

Used by — the kept funding book

crypto_strategies · model_stack (Python)

How it works — Sums the model votes for a symbol; when at least two agree (|net| ≥ 2) it trades the consensus — long if the net is positive, short if negative. It reads the alert and model-vote streams from the Go backend.

Best used for — A consensus filter: it only acts when several independent models point the same way, which is meant to cut the single-model noise that Stage-0 found costs the book.

Used by — the kept consensus book

churn (Python)

How it works — A throughput demo, honestly labelled: one fill per second regardless of direction, alternating side. It exists to prove the whole pipeline (signal → decision → fill → ledger) moves, not as an edge — at 0.2% round-trip its fees alone bleed the book.

Best used for — A canary for the pipeline, not a strategy. Its fills exercise the decision→fill→ledger chain so the substrate stays honest.

Used by — the pipeline canary (honestly not an edge)

backend · turtle / detectors (Go :8787)

How it works — The Go detector server: it consumes market flow/depth/ticks and runs the pump/dump onset and turtle-breakout detectors, emitting alerts plus the screen/survey surfaces.

Best used for — Its alert.pumpdump feed is the alert stream fast_ignition and crypto_strategies act on; the screen/survey surfaces are the owner's watch view.

Used by — fast_ignition + crypto_strategies — the alert feed

recorder (Go :8801)

How it works — The single bus consumer and the only writer of the lake. Every envelope passes the interceptor pipeline — authority, keying, correlation, freshness — each recorded as a hop, then is written once to a rotating, indexed, compressed segment.

Best used for — The durable record (state/lake3) every research artefact reads: the F4 replay, the solver, fit_weights and the decision journal all replay this lake. Nothing is ever deleted — segments seal, never truncate.

Used by — every research artefact — F4 replay, solver, fit_weights

portfolio_review (Python)

How it works — Grades every book (trade count, return_pct, recent-loss streak) and stops the underperformers, UNION-adding them to the disable list so it never clobbers the owner's config. Publishes each verdict.

Best used for — The book lifecycle: it removes books that have had enough trades to judge and still lose, and its review.verdict feed is the audit trail of those stops.

Used by — the book lifecycle — stops the underperformers

Chapter 6 · Books — the paper accounts

A **book** is one model's paper account: its **equity**, its **open positions**, and its **closed trades**. One book per model. The book is the only thing that remembers — the model re-scores every cycle, but the book accumulates the realised PnL that the leaderboard and the solver measure.

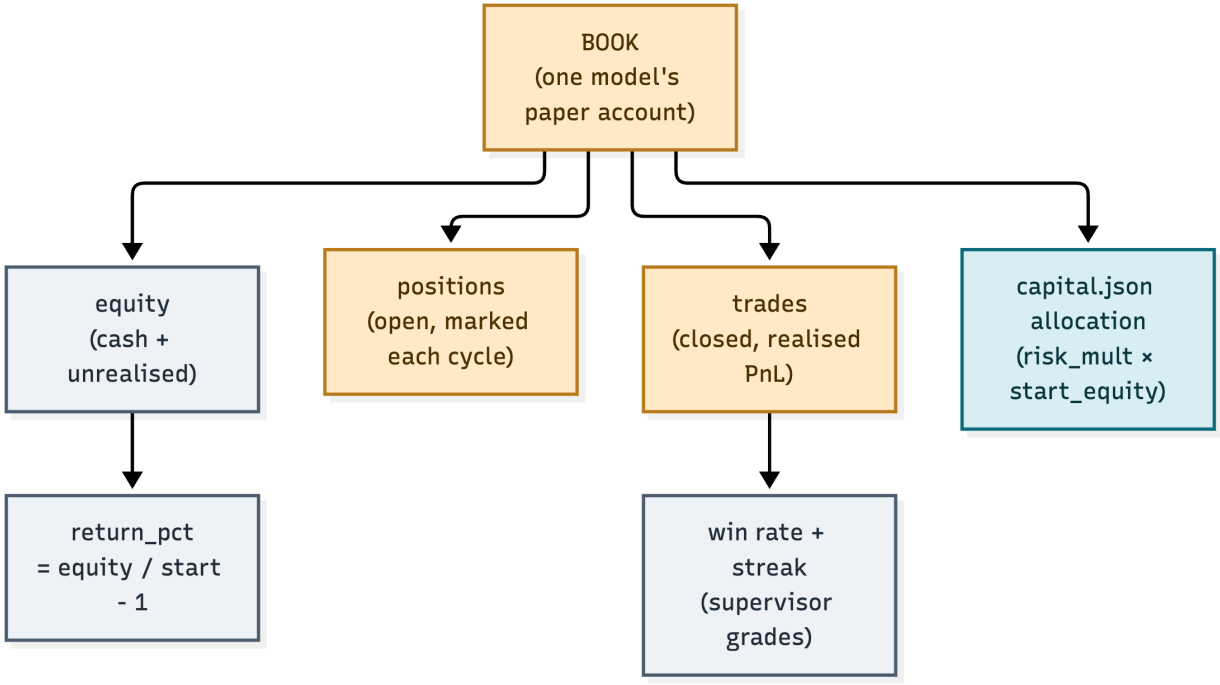


Figure 6.1 — a book's anatomy: equity, positions, trades, and the capital allocation that decides how much it may size.

Equity moves two ways: an open position is **marked to market** each cycle (equity drifts with the unrealised PnL), and a closed position realises PnL into equity permanently. `return_pct = equity / start_equity − 1` is the number the leaderboard shows, and `capital.json` stores each book's `risk_mult` — a promoted book may size up, a demoted one may only size down, and a disabled one gets zero allocation (A6).

6.1 · From signal to an open position

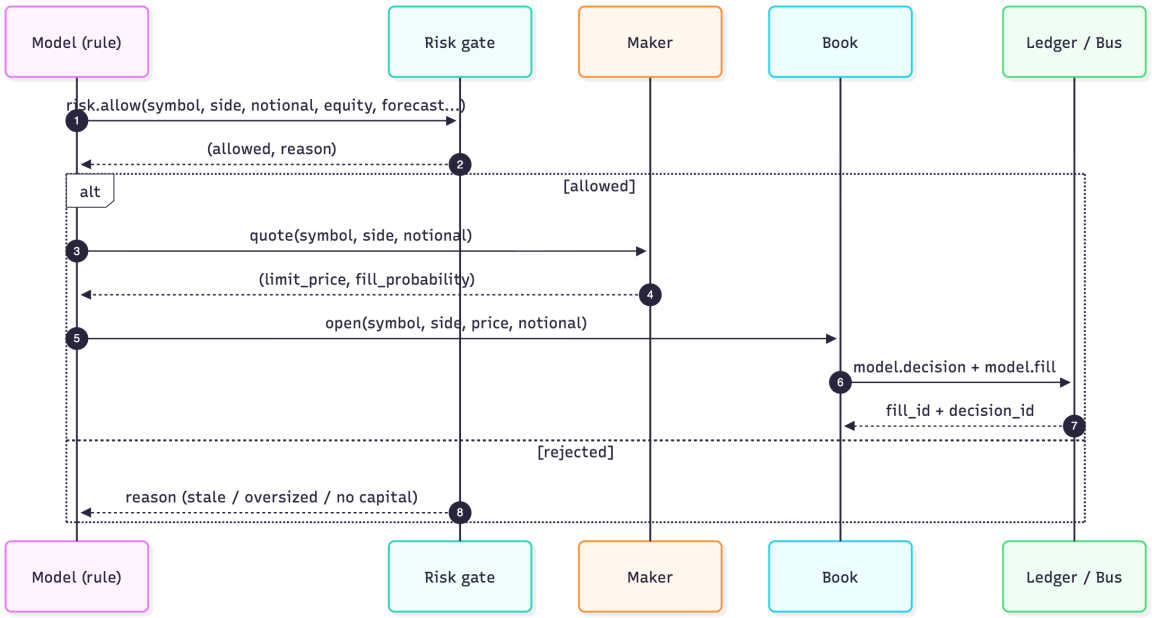


Figure 6.2 — the sequence: risk, then a maker quote, then the book opens, and the decision/fill are published with linked ids.

Notice the ordering: **risk before size, quote before open**. risk.allow refuses an oversized or stale entry; maker.quote returns a limit price that joins the book instead of crossing it; then the book opens and publishes model.decision + model.fill with linked decision_id / fill_id so the lake can thread the whole life of the position.

6.2 · The life of one position

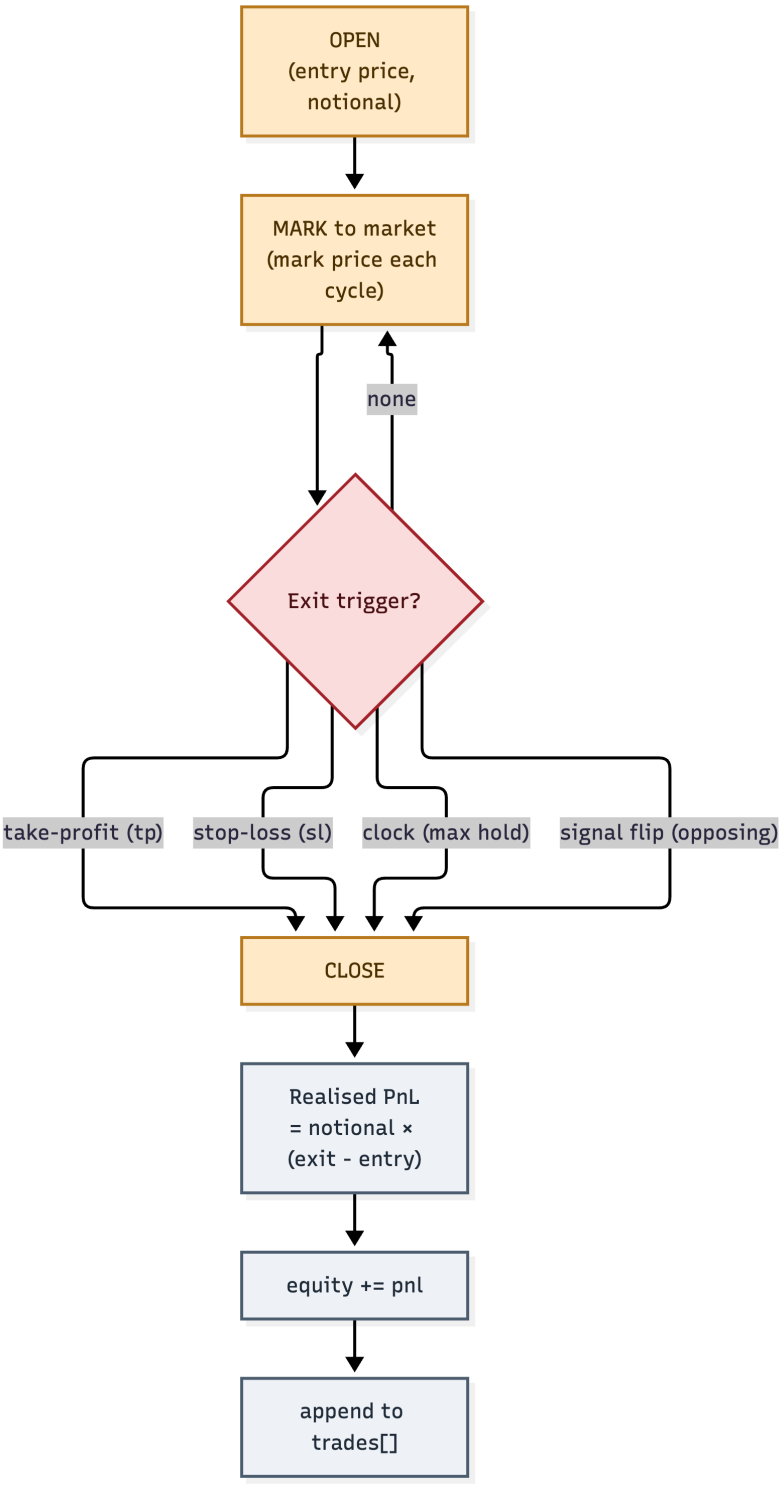


Figure 6.3 — a position is opened once, marked every cycle, and closed by one of four triggers; the realised PnL lands in equity and the trades list.

6.3 · How size is decided

Three multipliers stack, in order. First the model's **risk budget**: `risk_pct` of equity, capped at 8% (models10) or 3× (alpha), turned into a notional by the stop distance. Then **conviction sizing**: `risk_for(base, conf) = base × (0.6 + 0.2 × min(conf, 4))` — more confirmations, more capital. Then the global **size_mult** (now 3.0) scales the final notional. Shorts on models10 carry half the size (the unbounded-tail rule).

Chapter 7 · A signal becomes a trade, end to end

Everything in one sequence. Read top to bottom: market_data feeds the whole platform; tad, forecast and feeds each derive one input; the model scores; risk admits; the book opens; and the decision + fill are published back onto the bus for the lake and the dashboard.

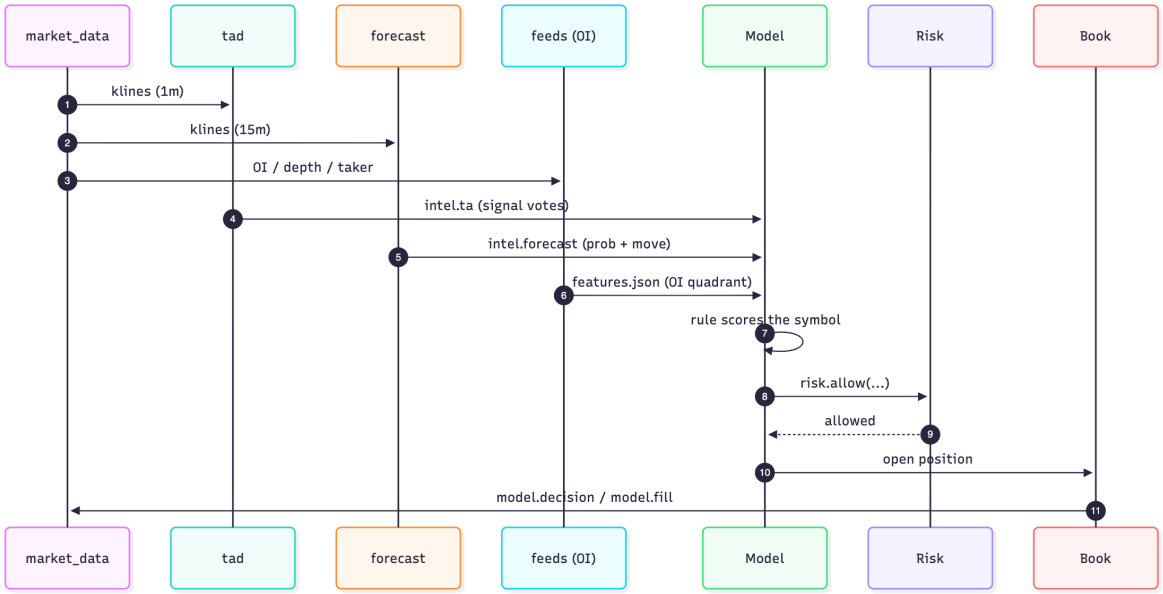


Figure 7.1 — the full signal-to-trade flow, one symbol.

7.1 · A worked example, in pseudo-code

```
# 1. inputs arrive on the bus (BTCUSDT)
klines = market_data.klines("BTCUSDT", "1m") # 120 bars
ta = tad.signals("BTCUSDT") # EMA/RSI/Donchian votes
fc = forecast.blend("BTCUSDT", "buy") # prob 0.61, move 0.4%
oi = feeds.quadrant("BTCUSDT") # "confirmation"

# 2. the model scores
side, score = funding_fade("BTCUSDT") # fr=+0.10% -> ("sell", 0.8)

# 3. the gates run
if fc.prob < 0.45: skip # 0.61 passes
score *= oi_gate.adjust(score, oi, side) # confirmation boosts the move
ok, why = risk.allow("BTCUSDT", side, notional, equity, fc.move_pct)
if not ok: skip # exposure/staleness gate

# 4. the trade opens
px, prob = maker.quote("BTCUSDT", side, notional) # join the bid
book.open("BTCUSDT", side, px, notional) # position live
emit.decision + emit.fill # linked ids -> lake
```

Chapter 8 · The message bus

8.1 · Bus architecture — how many of each

Layer	Count	Description
Broker	1	RabbitMQ, STOMP :61613 — the single message backbone
Exchanges	8	market, intel, alert, model, risk, review, exec, system (routing groups)
Topics	22	routing keys within the exchanges (the full registry above)
Queues	~60	2 per topic (q. for bus_consumer, q.rec. for recorder) + stomp-subscription-* for live subscribers
Producers	14	market_data, tad, forecast, signals, feeds, market_agent, portfolio_review, books, backend, supervisor, healthpub, master, cli, maker
Consumers	5	recorder (:8801), bus_consumer (:8798), backend (:8787), signal_bus (:8793), master (:8799)
Secondary bus	1	signal_bus :8793 is an HTTP proxy (/latest) the books poll — not a second RabbitMQ broker

8.2 · Registry — the 22 topics

Topic	Exch	Ret	Publisher(s)	Other subscriber(s)
`market.price`	market	drop	market_data	backend (turtle), master
`market.candle`	market	drop	market_data	tad (TA), forecast (warm-up)
`market.depth`	market	drop	market_data	backend (detectors), feeds
`market.mini24h`	market	drop	market_data	master (24h change)
`ticks`	market	drop	market_data	backend (flow windows)
`intel.signal`	intel	60s	signals	books (crypto_pack / executor / alpha via :8793)
`intel.flow`	intel	60s	signals	books (flow confidence)
`intel.movers`	intel	60s	market_data, signals	books (movers surface)
`intel.board`	intel	60s	market_agent	books, master (opportunity board)
`intel.ta`	intel	60s	tad	books (ta_vote via ta_live.json)
`intel.forecast`	intel	60s	forecast	books (forecast gate)
`alert.pumpdump`	alert	300s	backend, signal_bus	fast_ignition, crypto_strategies
`model.decision`	model	5m	alpha_rank, models10, crypto_pack, executor, alpha_signals, skim_pump, curve_rider, hft, movers, grid	master (journal)
`model.fill`	model	5m	same as decision	master, fit_weights (research)
`model.pnl`	model	5m	decision producers + supervisor	master, capital (allocator)
`model.rank`	model	5m	alpha_rank	master
`model.evaluate`	model	60s	tad	books (signal-completion pacer)
`risk.state`	risk	5m	risk	books (risk gate)
`review.verdict`	review	5m	portfolio_review	master
`exec.quote`	exec	60s	maker	
`system.health`	system	10m	healthpub, master	master
`system.control`	system	1d	master, supervisor, cli	master

8.3 · Producer contract — what each model sends and where it is inserted

Wire format: Python producers pass the registry gate (topic registered AND producer in its allow-list), then wrap the payload in an envelope `{topic, key, producer, data, version:1, seq, ts}` and SEND it to `/exchange/<exchange>/<topic>` on RabbitMQ :61613 (lib/bus.py:101-126,176-198). Go producers send the raw payload body (marketdata adds `v, plane, topic, producer, key, gen, seq, cid, computed_at` headers, stomp.go:183-204); Python consumers wrap it on receipt when the body has no topic field (bus.py:307-325). The exchange fans each message out to every bound queue — the recorder's durable mirror, bus_consumer's projection, and each live subscriber's private queue.

Topic	What the model produces	Data in the envelope	Inserted at (publish call sites)
`market.price`	Last trade price plus 24h-change metadata for the whole liquid universe, one compact map per tick.	`{p: {sym: px}, m: {sym: {c: pct24h, q: quoteVol}}, ts, msgs} — Go variant: {p, ts}`	`market_data.py:292` `core/cmd/marketdata/main.go:228`
`market.candle`	Closed 1m/15m OHLCV bars only (k.K.X) — the TA and forecast warm-up feed.	`{symbol, interval, t, o, h, l, c, v, q, Q, ts}`	`market_data.py:249` `core/cmd/marketdata/main.go:589→426`
`market.depth`	5-level order book per symbol with sizes, spread and bid/ask imbalance; published on change or 1s heartbeat.	`py: {symbol, bids: [[p,q]×5], asks: [[p,q]×5], spread_bps, bid_qty, ask_qty, imbalance, ts} — Go: bids/asks as [{price,qty}]`	`market_data.py:328-333` `core/cmd/marketdata/main.go:243`
`market.mini24h`	Universe-size heartbeat — how many symbols the 24h mini-ticker window covers.	`{count, ts}`	`market_data.py:246` `core/cmd/marketdata/main.go:229-230`

`ticks`	All-symbol tick snapshot (price + 24h move + quote vol); the flow-window input on the backend.	`{p: {sym: px}, m: {sym: {pct, qv, p}}, ts} — backup path: {p, ts, msgs} raw, no envelope`	`market_data.py:293`
			`core/cmd/marketdata/main.go:227`
			`market_data_rest_backup.py:304 (raw, bypasses registry)`

`intel.signal`	signals.py per-symbol analysis score — taker-buy ratio, flow confidence and parts blended into one rankable number.	`{symbol, taker_by_ratio, flow_conf, flow_parts, score, ts} (from analyse())`	`signals.py:325`
`intel.flow`	Flow confidence per symbol with the last 10 flow readings — the books' flow-conviction input.	`{symbol, buy_ratio, flow_conf, flow_series: last 10, score}`	`signals.py:326-332`
`intel.movers`	Gainers/losers/most-traded surface. Two producer variants with different payload shapes.	`signals: {gainers, losers, most_traded, universe, ts} — market_data (slimmer): {gainers: [{s,pct,qv}]×15, losers, ts}`	`signals.py:306` `market_data.py:269-271`
`intel.board`	The market_agent opportunity board: top-15 ranked rows with score components, its natural-language thoughts, and flagged symbols. Owner-critical.	`{ts, universe, opportunities: rows[:15], thoughts: [{kind,sym,score,text,ts}][:20], flagged: [sym]}`	`market_agent.py:127`

`intel.ta`	tad per-symbol TA snapshot — the full signal set per cycle, or a single firing signal.	`{symbol, signals: [sig], count, ts} or {signal: sig, ts}; sig = {cursor, symbol, kind, side, strength, note, ts}`	`tad/main.go:317`
			`tad/main.go:326`
			`tad/main.go:414 (/scan)`

`intel.forecast`	forecast service blended forecast per symbol+side+horizon with agent views and a trade action (enter/hold/liquidate). Gates book entries.	`{symbol, side, horizon_s, prob, move_pct, confidence, action, resolution, views: [{agent, side, prob, move_pct, confidence, note}], price, bar_age_s, ts}`	`forecast/main.go:502` `forecast/main.go:565`

Topic	What the model produces	Data in the envelope	Inserted at (publish call sites)
alert.pumpdump	Pump/dump alert from the backend detectors with severity, reasons and full metrics; signal_bus republishes with cursor+received_ms.	{symbol, kind, score, event_id, severity, summary, ts_ms, detector, reasons: []} + {cursor, received_ms} on republish	signal_bus.py:33-50 core/cmd/signalbus/main.go:75
model.decision	A book's entry decision, chained by decision_id to every later fill. Emitted through the single lib/emit emitter by every book that opens a position.	{decision_id, engine, symbol, side, allowed, reason, confidence, ts_ms}	lib/emit.py:85 ← models10.py:268, crypto_strategies.py:222, movers.py:263, hybrids.py:287, alpha_rank.py:172, grid.py:218, churn.py:76, hft.py:280, alpha_signals.py:363
model.fill	A closed position's result, carrying fill_id + the originating decision_id — the decision-outcome substrate.	{fill_id, decision_id, engine, symbol, side, entry, exit, gross_pct, net_pct, peak_pct, reason, held_s, notional, pnl_usd, entry_ms, exit_ms, features, rank_score, why, slippage_bps, signal_price}	lib/emit.py:61 ← models10.py:299, crypto_strategies.py:173, 213, movers.py:230, hybrids.py:311, alpha_rank.py:125, grid.py:253, churn.py:66, hft.py:249, alpha_signals.py:180,223,272
model.pnl	Equity mark: alpha_rank on each position close; the supervisor on each cycle for every graded book.	alpha_rank: {engine, equity, net_pct, symbol} — supervisor: {engine, equity, return_pct, trades, open}	alpha_rank.py:111-113 supervisor.py:317-320
model.rank	alpha_rank cross-sectional ranking — top and bottom of the liquid universe by blended score.	{top: [{symbol, score}]×8, bottom: [...]×8, ts} (key 'xsec')	alpha_rank.py:134-137
model.evaluate	tad's bar-close pacer pulse with a fresh feature snapshot — the book-decision layer's completion signal. Currently NO subscriber.	{symbol, reason: 'bar_close', ts_ms, inputs: fresh.snapshot(sym, now)}	tad/main.go:363
risk.state	risk.py portfolio snapshot — gross exposure, direction counts, per-symbol notionals; the shared gate's view of the books.	{snapshot: {equity, start_equity, return_pct, gross_notional, gross_multiple, longs, shorts, per_symbol: {sym: notional}}, ts}	risk.py:185
review.verdict	portfolio_review's per-book verdict each cycle — stop for evidence-losing books, keep for the rest. Its disable write is a read-modify-UNION (A6).	{engine, action: stop|keep, why, return_pct}	portfolio_review.py:25-34 (pub fn) portfolio_review.py:102 (call)
exec.quote	maker.py quote intent — the paper execution layer's priced quote per symbol+side.	{symbol, side, px, prob, notional}	maker.py:49-50
system.health	Platform health snapshot — every service's up/down, latency and port; the dashboard's health panel source.	{services: [{name, up, ms, detail?, port}], up, total, worst_ms, ts}	health_pub.py:70 core/cmd/healthpub/main.go:97
system.control	Control plane for operator commands (restart/stop/flag). WARNING: the only publish site is malformed (TypeError swallowed at master.py:241; Bus has no close()) — the topic is effectively unsent.	{action, actor: 'dashboard', ts} — intended shape	master.py:241 (BROKEN — never sends)

8.4 · Message flow — how a model's output travels

A model (book) computes a result and publishes it to **one topic**. The bus wraps it in an **envelope** and routes it by exchange + routing key. Every consumer receives the envelope and reads **meta.key** (the symbol) to act on the right instrument — routing is by key, never by broadcast.

Aspect	Envelope fields
The model REQUIRES (reads from an inbound envelope)	meta.key (the symbol), meta.topic, data (the payload) — that is all a consumer needs to route and act
The model ADDS (when it publishes)	producer (its name), data (its calculation), key (the symbol), plus CID / refs (the decision→fill provenance)
The model COMMUNICATES BACK	one topic it publishes to (e.g. model.decision / model.fill) carrying the payload

Pseudo-code

```
# 1 · a model computes a result
side, conf = funding_fade_strategy(klines("BTCUSDT")) # -> ("buy", 0.7)

# 2 · it publishes ONE result to ONE topic, tagged with the symbol
emit.decision("models10", "BTCUSDT", "buy", True, "funding_fade")
-> bus.pub("model.decision", "BTCUSDT", payload, producer="models10")
-> spec = TopicRegistry["model.decision"] # exchange "model"
-> guard: producer "models10" must be in spec.producers # else REFUSED
-> env = Envelope(topic="model.decision", key="BTCUSDT",
producer="models10", data=payload) # stamps version+received_at
-> transport.publish(exchange="model", routing_key="model.decision", env.json)

# 3 · the broker fans out to the queues
# q.model.decision (bus_consumer) · q.rec.model.decision (recorder)
# stomp-subscription-* (master + other live subscribers)

# 4 · the recorder runs the interceptor pipeline, then writes the lake
for rule in [authority, keying, correlation, freshness]:
env = rule(env); env.meta.hops.append(hop(rule, outcome)) # pass | refuse

# 5 · the correct engine receives it and routes by key (the symbol)
def on_model_decision(env):
sym = env.meta.key # "BTCUSDT" -> the engine acts on THAT symbol
data = env.data # the decision payload
book.consider(sym, data) # no broadcast; the key selects the symbol
```

8.5 · Subscriber breakdown — every topic, every subscriber

recorder (:8801) and **bus_consumer (:8798)** subscribe to every topic; the breakdown lists them per topic plus each topic-specific consumer with its usage and code location.

Topic	Subscriber	Usage	Code (file:line)
`market.price`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	backend (turtle)	turtle book pricing	`backend/internal/turtle/store.go`
`market.candle`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	tad	TA bars (1m)	`tad/main.go:62`
	forecast	warm-up bars	`forecast/main.go:89`
`market.depth`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	backend (detectors)	depth/pump detectors	`backend/internal/detect/*.go`

Topic	Subscriber	Usage	Code (file:line)
	feeds	book imbalance	`feeds.py:74`
`market.mini24h`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	24h change surface	`master.py`
`ticks`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	backend (flow)	flow windows	`backend/internal/flow/*.go`
`intel.signal`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	entry signals via :8793 /latest	`crypto_pack.py:182`, `executor.py:41`
`intel.flow`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	flow confidence	`crypto_pack.py:182`
`intel.movers`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	movers surface	`crypto_pack.py:182`
`intel.board`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books / master	opportunity board	`market_agent.py:126 (pub)`
`intel.ta`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	tactics.ta_signals	TA vote → ta_live.json	`tactics.py:362`
`intel.forecast`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	forecast gate	`tactics.py forecast()`
`alert.pumpdump`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	fast_ignition	alert-scored entries	`fast_ignition.py`
	crypto_strategies	pump/dump signals	`crypto_strategies.py:12`
`model.decision`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	decision journal	`master.py:1009`
`model.fill`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	fill journal	`master.py:1009`
	fit_weights	research weights	`fit_weights.py`
`model.pnl`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	pnl surface	`master.py:1009`
	capital	allocator input	`capital.py`
`model.rank`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`

Topic	Subscriber	Usage	Code (file:line)
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	rank surface	`master.py:1009`
`model.evaluate`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	signal-completion pacer pulse	`tad/main.go:320 (pub)`
`risk.state`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	books	risk gate	`risk.py`
`review.verdict`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	review surface	`master.py:1009`
`exec.quote`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
`system.health`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	health panel	`master.py:7239`
`system.control`	recorder (:8801)	single lake writer (all topics)	`core/cmd/recorder/main.go:222`
	bus_consumer (:8798)	record projection (all topics)	`bus_consumer.py:64`
	master	control commands	`master.py`

Chapter 9 · Known gaps — models not listening

These models read a **HTTP/file proxy** instead of the bus topic, or the topic has **no consumer**. The bus carries the topic; the model never subscribes to it.

Model	Should listen to	Gap	Code (file:line)
all books	`model.evaluate`	the signal-completion pacer pulse (published by tad) has NO subscriber — the book-decision layer is unpaced	`tad/main.go:320 (pub) — no consumer`
alpha (funding_carry, xsec_momentum)	`intel.board`	reads the opportunity board via master /api/opportunities (file proxy), not the intel.board topic	`alpha.py:175`
crypto_strategies (model_stack)	`alert.pumpdump`	reads /api/alerts from backend :8787 (HTTP), not the alert.pumpdump topic	`crypto_strategies.py:12`
fast_ignition (disabled)	`alert.pumpdump`	reads the alert stream via backend HTTP, not the topic	`fast_ignition.py`
models10 (funding_fade)	`intel.ta`	reads TA via ta_live.json (tactics.ta_signals), not the intel.ta topic	`tactics.py:362`
models10 (funding_fade)	`intel.forecast`	reads forecast via FC_API :8790, not the intel.forecast topic	`models10.py:66`
crypto_pack / executor (disabled)	`intel.signal / intel.flow / intel.movers`	the signal consumers are disabled under best-5, so these topics have no live subscriber	`crypto_pack.py:182, executor.py:41`